http://pubsonline.informs.org/journal/opre/

OPERATIONS RESEARCH
Vol. 67, No. 2, March–April 2019, pp. 532–547
ISSN 0030-364X (print), ISSN 1526-5463 (online)

On the Minimum Chordal Completion Polytope

David Bergman,[a] Carlos H. Cardonha,[b] Andre A. Cire,[c] Arvind U. Raghunathan[d]

[a] Operations and Information Management, University of Connecticut, Storrs, Connecticut 06260; [b] IBM Research, São Paulo 04007-900, Brazil; [c] Department of Management, University of Toronto Scarborough and Rotman School of Management, Toronto, Ontario M1E-1A4, Canada; [d] Mitsubishi Electric Research Labs, Cambridge, Massachusetts 02139

Contact: david.bergman@business.uconn.edu, http://orcid.org/0000-0002-5566-5224 (DB); carloscardonha@br.ibm.com, http://orcid.org/0000-0002-1439-5205 (CHC); andre.cire@utoronto.ca, http://orcid.org/0000-0001-5993-4295 (AAC); raghunathan@merl.com, http://orcid.org/0000-0003-3173-387 (AUR)

Received: August 19, 2016
Revised: August 21, 2017; February 9, 2018
Accepted: May 18, 2018
Published Online in Articles in Advance: March 21, 2019

Subject Classifications: programming: integer, algorithms, cutting plane/facet; analysis of algorithms: computational complexity
Area of Review: Optimization

https://doi.org/10.1287/opre.2018.1783

Abstract. A graph is chordal if every cycle with at least four edges contains a *chord*—that is, an edge connecting two nonconsecutive vertices of the cycle. Several classical applications in sparse linear systems, database management, computer vision, and semidefinite programming can be reduced to finding the minimum number of edges to add to a graph so that it becomes chordal, known as the *minimum chordal completion problem* (MCCP). We propose a new formulation for the MCCP that does not rely on finding perfect elimination orderings of the graph, as has been considered in previous work. We introduce several families of facet-defining inequalities for cycle subgraphs and investigate the underlying separation problems, showing that some key inequalities are NP-hard to separate. We also identify conditions through which facets and inequalities associated with the polytope of a certain graph can be adapted in order to become facet defining for some of its subgraphs or supergraphs. Numerical studies combining heuristic separation methods and lazy-constraint generation indicate that our approach substantially outperforms existing methods for the MCCP.

Funding: The research was partially funded by a Natural Sciences and Engineering Research Council Discovery grant.
Supplemental Material: The online supplement is available at https://doi.org/10.1287/opre.2018.1783.

Keywords: networks/graphs • applications: networks/graphs • programming: integer • algoritms: cutting plane/facet

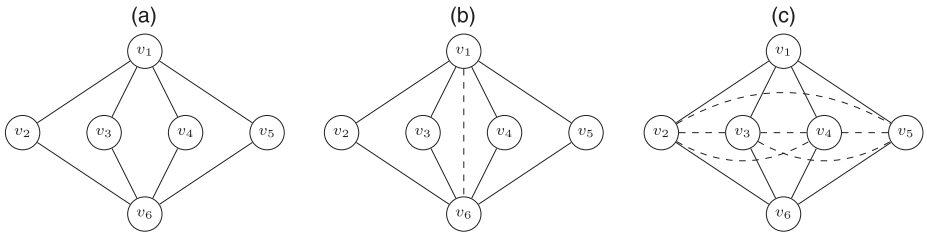
1. Introduction

Given a simple graph $G = (V, E)$, the *minimum chordal completion problem* (MCCP) asks for the minimum number of edges to add to E so that the graph becomes *chordal*; that is, every cycle of length at least four in G has an edge connecting two nonconsecutive vertices (i.e., a *chord*). Figure 1(b) depicts an example of a minimum chordal completion of the graph in Figure 1(a), where a chord $\{v_1, v_6\}$ is added because of the six chordless cycles (v_1, v_i, v_6, v_j) for all pairs of $i,\ j$ with $2 \leq i < j \leq 5$. The problem is also referred to as the *minimum triangulation problem* or the *minimum fill-in problem*.

The MCCP is a classical combinatorial optimization problem with a variety of applications spanning both the operations research and the computer science literature. Initially motivated by problems arising in Gaussian elimination of sparse linear equality systems (Parter 1961), chordalization methods have established an active research field, with applications in database management (Beeri et al. 1983, Tarjan and Yannakakis 1984), sparse matrix computation (Grone et al. 1984, Fomin et al. 2013), artificial intelligence (Lauritzen and Spiegelhalter 1990), computer vision (Chung and Mumford 1994), and in several other contexts (Heggernes 2006). Most recently, heuristic solution methods for the MCCP have gained a central role in semidefinite and nonlinear optimization, in particular for exploiting sparsity of nonlinear constraint matrices (Nakata et al. 2003, Kim et al. 2011, Vandenberghe and Andersen 2015).

The literature on exact computational approaches for the MCCP is, however, surprisingly scarce. To the best of our knowledge, the first mathematical programming model for the MCCP is derived from a simple modification of the formulation by Feremans et al. (2002) for determining the *tree-widths* of graphs. That model is based on a result by Fulkerson and Gross (1965), stating that a graph is chordal if and only it has a *perfect elimination ordering*, which is an ordering of vertices such that any vertex v forms a clique with its succeeding neighbours in the ordering. Yüceoğlu (2015) recently provided the first polyhedral analysis and computational testing of this formulation, deriving facets and other valid inequalities for specific classes of graphs. Alternatively, Bergman and Raghunathan (2015) introduced a Benders approach to the MCCP that relies on a simple class of valid inequalities, outperforming a backtracking search algorithm. In all such cases, theoretical results have focused on specific families of graphs, and several benchmark graphs with fewer than 30 vertices were still not solved to optimality within reasonable computational times.

Figure 1. Examples of Chordal Completions



Notes. (a) An example graph. (b) A minimum chordal completion of the graph. (c) A minimal, nonoptimal chordal completion of the graph.

1.1. Our Contributions

In this paper, we investigate a novel mathematical programming model for the MCCP that extends the preliminary work of Bergman and Raghunathan (2015). Our formulation is composed of exponentially many constraints—the *chordal inequalities*—that are defined directly on the edge space of the graph and do not depend on the perfect elimination ordering property, as opposed to earlier formulations. We investigate the polyhedral structure of such a model, which reveals that the proposed inequalities are part of a special class of constraints that induce exponentially many facets for cycle subgraphs. This technique can be generalized to lift other inequalities and strengthen the corresponding linear programming relaxation. In particular, we propose three additional families of valid inequalities and provide a study on the computational complexity of the associated separation problems.

Building on these theoretical results, we propose a hybrid solution method for the MCCP that alternates a lazy-constraint generation with a heuristic separation procedure. The resulting approach is compared with the current state-of-the-art models in the literature, and it is empirically shown to improve solution times and known optimality gaps for standard graph benchmarks, often by orders of magnitude. Our numerical results also indicate that the optimal completions can be significantly smaller than the ones provided by state-of-the-art heuristic methods.

1.2. Organization of the Paper

We start by discussing previous works related to the MCCP in Section 2 and introducing our notation in Section 3. In Section 4, we describe a new integer programming (IP) formulation for the MCCP and characterize its polytope in Section 5, also proving dimensionality results and simple upper bound facets. Section 6 provides an in-depth analysis of the polyhedral structure of cycle graphs, introducing four classes of facet-defining inequalities. In Section 7, we prove general properties of the polytope, including results concerning lifting of facets. Finally, in Section 8, we propose a hybrid solution technique that considers both a lazy-constraint generation and a heuristic-separation method based on a threshold-rounding procedure, and also present a simple primal heuristic for the problem. We provide a numerical study in Section 9, indicating that our approach substantially outperforms existing methods, in particular solving many benchmark graphs to optimality for the first time.

2. Previous Works

The first graph-theoretical aspects of the MCCP were introduced by Parter (1961) and Rose (1972) in the context of sparse matrix computations. The authors simulated the Gaussian elimination process of a linear system of equations as an *elimination game* in a graph constructed according to the coefficient matrix of the system. In such a game, vertices are removed one at a time and, for each removed vertex v, edges are added so that v forms a clique with its neighbors in the remaining graph. Each new edge corresponds with adding a nonzero entry (i.e., a *fill-in*) to the original matrix, which impacts the total time and storage requirements of the Gaussian elimination process. The total number of edges added, in turn, was shown to be heavily dependent on the order in which vertices are removed during the elimination game.

The MCCP was then formally proposed by Rose et al. (1976), Ohtsuki (1976), and Ohtsuki et al. (1976) simultaneously. This formalization builds on a result by Fulkerson and Gross (1965), which indicates that the family of chordal graphs coincides with the graphs produced by the elimination game. That is, finding an ordering with minimum fill-in corresponds to adding the minimum number of edges so that the graph becomes chordal. Rose et al. (1976) and Ohtsuki (1976) also proposed the first algorithms for finding *minimal* chordalizations in linear time on the size of the graph. A follow-up work by Rose and Tarjan (1978) proved that the MCCP was NP-Hard for directed graphs, but the undirected case was only stated as a conjecture. Later, Garey and Johnson (1979) posed the MCCP as problem #4 among the 12 major open problems in computational complexity, sparking new studies on both theoretical and computational aspects of the problem. The complexity question was settled by Yannakakis (1981), who showed that the MCCP was NP-hard

using a reduction from the optimal linear arrangement problem.

In regards to the theoretical aspects of the problem, a large stream of research has focused on alternative characterizations based on *graph separators* as opposed to vertex orderings. A (u, v)-graph separator is a vertex subset that disconnects vertices u and v when removed. Kloks et al. (1993) showed that, if the set of minimal separators of a graph G can be enumerated efficiently, the MCCP and the related *tree-width* problem are solvable in polynomial time. This includes a large variety of graph classes such as permutation graphs, trapezoid graphs, and circle graphs, to name a few. Parra and Scheffler (1995, 1997) and Kloks et al. (1999), with results later completed by Kratsch and Müller (2009), demonstrated that one could obtain a minimal chordal completion by obtaining a maximal set of parallel (i.e., noncrossing) separators and completing them into cliques. Finally, the influential work by Bouchitté and Todinca (2001) provided key results concerning these questions. Namely, it showed that the MCCP is tractable in polynomial time if the set of *potential maximal cliques*—that is, vertex subsets that induce cliques in some minimal chordalization, could be listed in polynomial time. The result also answers a conjecture posed by Kloks et al. (1993), showing that the MCCP is solvable efficiently for *weakly triangulated graphs*—that is, graphs with a polynomial number of separators (not necessarily minimal), which generalizes previous work considering specialized graphs (Chang 1996, Kloks et al. 1998).

There also has been an extensive literature on approximation algorithms and fixed-parameter tractable (FPT) procedures for the MCCP. Agrawal et al. (1993) presented the first polynomial-time algorithm with a guaranteed approximation ratio specifically dependent on the size of the graph and on the maximum number of edges k that can be added. Natanzon et al. (2000) later proposed a new algorithm with an approximation ratio of $\mathcal{O}(8k^2)$—that is, at most $8k$ from the optimal solution. The first FPT algorithm was proposed by Kaplan et al. (1999), with complexity exponential on k. Fomin and Villanger (2012) later proposed a substantially faster FPT that was subexponential on k and based on the results by Bouchitté and Todinca (2001). Most recently, Bliznets et al. (2016) prove lower bounds on the complexity of FPTs under the Exponential Time Hypothesis. Cao and Sandeep (2016) and Cao and Marx (2016) extend these results by producing new parameterized lower bounds, while also excluding possible approximation schemes through a new reduction from vertex cover.

For computational approaches, the primary focus has been on heuristic methodologies with no optimality guarantees, including techniques by Mezzini and Moscarini (2010), Rollon and Larrosa (2011), and Berry et al. (2003, 2006). The state-of-the-art heuristic, proposed by George and Liu (1989), is a simple and efficient algorithm based on ordering the vertices by their degree. Previous articles have also developed methodologies for finding a *minimal* chordal completion. Note that a minimal chordal completion is not necessarily *minimum*, as depicted in Figure 1(c), but does provide a heuristic solution to the MCCP.

Finally, exact computational methodologies for the MCCP have been limited, to the best of our knowledge. A valid mathematical programming model for the MCCP can be obtained by modifying the formulation by Feremans et al. (2002) for determining the *tree-widths* of graphs based on perfect elimination orderings. Yüceoğlu (2015) has recently provided a first polyhedral analysis and computational testing of this formulation, deriving facets and other valid inequalities for specific classes of graphs. Alternatively, Bergman and Raghunathan (2015) introduced a Benders approach to the MCCP, outperforming a simple constraint programming backtracking search algorithm.

We note in passing that the MCCP literature is extensive; we refer to the survey by Heggernes (2006) for additional details and references. We contribute to this body of work by investigating the polyhedral structure of a new mathematical programming formulation for the problem. Besides the underlying structural results from this study, we derive a computational approach that has been effective in providing new solutions and improved bounds for benchmark problems.

3. Notation and Terminology

For the remainder of the paper, we assume that each graph $G = (V, E)$ is connected, undirected, and does not contain self-loops or multiedges. For any set S, $\binom{S}{2}$ denotes the family of two-element subsets of S. Each edge $e \in E \subseteq \binom{V}{2}$ is a two-element subset of vertices in V. The *complement edge set* (or *fill edges*) E^c of G is the set of edges missing from G; that is, $E^c = \binom{V}{2} \backslash E$. We denote by m and m^c the cardinality of the edge set and of the complement edge set of G, respectively (i.e., $m = |E|$ and $m^c = |E^c|$). The graph *induced* by a set $V' \subseteq V$ is the graph $G[V'] = (V', E')$ whose edge set is such that $E' = E \cap \binom{V'}{2}$. If multiple graphs are considered in a context, we include "(G)" in the notation to avoid ambiguity; for example, $V(G')$ and $E(G')$ represent the vertex and edge set of a graph G', respectively. Moreover, for every integer $k \geq 0$, we let $[k] := \{1, 2, \ldots, k\}$.

For any ordered list $C = (v_0, v_1, \ldots, v_{k-1})$ of k distinct vertices of V, let $V(C) = \{v_0, v_1, \ldots, v_{k-1}\}$ be the set of vertices composing C, with $|C| = |V(C)|$. The *exterior* of C is the family

$$\xi(C) = \{\{v_{k-1}, v_0\}\} \cup \bigcup_{i \in [k-1]} \{\{v_{i-1}, v_i\}\},$$

and the *interior* of C is the family of two-element subsets of $V(C)$ that do not belong to $\xi(C)$,

$$\iota(C) = \binom{V(C)}{2} \backslash \xi(C).$$

An ordered list C is a *cycle* if $\xi(C) \subseteq E$. If C is a cycle, an element of $\iota(C)$ is referred to as a *chord*. A cycle C for which the induced graph $G[V(C)]$ contains no chords is a *chordless* cycle. G is said to be *chordal* if the maximum size of any chordless cycle is three. A chordless cycle with k vertices is denoted by *k-chordless cycle*.

Let $G = (V, E)$. Any subset of fill edges $F \subseteq E^c$ is a *completion* of G, and $G \cup F$ represents the graph that results from the addition of edges in F to E; that is, $G \cup F := (V, E \cup F)$. A *chordal completion* of G is any set of fill edges $F \subseteq E^c$ for which the completion $G \cup F$ is chordal. A *minimal chordal completion* F is a chordal completion such that, for any proper subset $F' \subset F$, F' is not a chordal completion of G. A *minimum chordal completion* is a minimal chordal completion of minimum cardinality, and the minimum chordal completion problem is concerned with the identification of a minimum chordal completion.

Example 1. The graph in Figure 1(a) has six chordless cycles, Figure 1(c) shows a chordal completion consisting of edges $\{v_2, v_3\}$, $\{v_2, v_4\}$, $\{v_2, v_5\}$, $\{v_3, v_4\}$ $\{v_3, v_5\}$, and $\{v_4, v_5\}$. Removing any of these edges will result in a graph that is not chordal, and hence this chordal completion is minimal. Figure 1(b) shows a *minimum* chordal completion consisting only of edge $\{v_1, v_6\}$.

4. IP Formulation for the MCCP

We now describe the IP formulation investigated in this paper. Given a graph $G = (V, E)$, each binary variable x_f in our model indicates whether the fill edge $f \in E^c$ is part of a chordal completion for G. That is, if we define the set $E(x) := \{f \in E^c : x_f = 1\}$ for each vector $x \in [0, 1]^{m^c}$, the set of feasible solutions to our model is given by

$$X(G) := \left\{x \in \{0, 1\}^{m^c} : G \cup E(x) \text{ is chordal}\right\}.$$

Thus, each $x \in X(G)$ equivalently represents the characteristic vector of a chordal completion of G. We use $G(x)$ to denote $G \cup E(x)$—that is, $G(x) = (V,\ E \cup E(x))$—and $x(F)$ to represent the characteristic vector of completion $F \subseteq E^c$—that is, $x(F)_f = 1$ if $f \in F$ and $x(F)_f = 0$ otherwise.

Let $\mathcal{C}$ be the family of all possible ordered lists composed of distinct vertices of V—that is, every $C \in \mathcal{C}$ can be written as a sequence $C = (v_0, v_1, \ldots, v_{k-1})$ for some $k \leq |V|$. Also, let $F_G(C) = F(C) \subseteq E^c$ be the set of fill edges that are missing in $\xi(C)$ for C to induce a cycle in G; that is,

$$F(C) := \xi(C) \backslash E(G).$$

We propose the following model for the MCCP:

$$\text{minimize} \quad \sum_{f \in E^c} x_f \qquad \text{(IPC)}$$

$$\text{s.t.} \quad \sum_{f \in \iota(C)} x_f - (|C| - 3)\left(\sum_{f \in F(C)} x_f - |F(C)| + 1\right) \geq 0, \quad \text{for all } C \in \mathcal{C}, \text{ with } \iota(C) \cap E = \emptyset$$

$$x \in \{0, 1\}^{m^c}. \qquad \text{(I1)}$$

The set of Inequalities (I1) will also be referred to as the *chordal inequalities*. Note that every sequence C such that $F(C) = \emptyset$ and $\iota(C) \cap E = \emptyset$ describes a cycle in G, and its associated Inequality (I1) simplifies to

$$\sum_{f \in \iota(C)} x_f \geq |C| - 3.$$

The following lemma is a reinstatement of fact 2.1 in Natanzon et al. (2000). It shows that Inequalities (I1) are valid in this special case.

Lemma 1. *If C is a chordless cycle of G such that $|C| \geq 4$, then any chordal completion of G contains at least $|C| - 3$ edges that belong to $\iota(C)$.*

Based on the previous lemma, we show below that the set of Inequalities (I1) is valid for all sequences in $\mathcal{C}$ and, as a consequence, that (IPC) is a valid formulation for the MCCP.

Proposition 1. *The model (IPC) is a valid formulation for the MCCP.*

Proof. We first show that there is an one-to-one correspondence between $X(G)$ and feasible points of (IPC). Let x^* be a feasible solution to (IPC) and suppose that the graph $G \cup E(x^*)$ contains a chordless cycle C with more than three vertices. Then $\sum_{f \in \iota(C)} x^*_f = 0 < |C| - 3$, thus contradicting the feasibility of x^*.

Conversely, let $E^* \subseteq E^c$ be such that $G \cup E^*$ is chordal, and suppose $x^* = \{x \in \{0, 1\}^{m^c} : x_f = 1 \Leftrightarrow f \in E^*\}$ is infeasible to (IPC)—that is, x^* violates at least one inequality of type (I1). Let C^* be a sequence associated with a violated inequality $I(C^*)$. We can assume that $|C^*| \geq 4$, as the inequality would be trivially satisfied otherwise. If $I(C^*)$ is violated, the expression multiplying $(|C^*| - 3)$ in the inequality must be equal to 1; otherwise, the sum would be less than or equal to 0, making the inequality trivially satisfied. Consequently, each edge in $\xi(C^*)$ must belong to $G \cup E^*$—that is, C^* is a cycle in $G \cup E^*$.

Moreover, by the definition of (I1), $\iota(C^*) \cap E = \emptyset$—that is, C^* is chordless in $G + F(C^*)$. Finally, a violation of $I(C^*)$ takes place if $\sum_{f \in \iota(C^*)} x_f < |C^*| - 3$, a condition that, according to Lemma 1, cannot hold if $G \cup E^*$ is chordal, so we have a contradiction. Finally, because the one-to-one correspondence holds and the objective function of (IPC) minimizes the number of added edges, the result follows. □

5. MCCP Polytope Dimension and Simple Upper Bound Facets

This section begins our investigation of the convex hull of the feasible set of chordal completions $X(G)$, which will lead to special properties that can be exploited by computational methods for the MCCP. We identify the dimension of the polytope and provide a proof that the simple upper bound inequalities $x_f \leq 1$ are facet defining.

Theorem 1. *If $G = (V, E)$ is not a complete graph (and hence not trivially chordal),*

a. $\text{conv}(X(G))$ *is full-dimensional;*
b. $x_f \leq 1$ *is facet-defining for all* $f \in E^c$.

Proof. We first show (a). Let $e \in \{0,1\}^{m^c}$ be the vector consisting only of ones and $e^j \in \{0,1\}^{m^c}$ be the unit vector for coordinate j. By definition, $G \cup E(e)$ is the complete graph, which is trivially chordal. By removing the edge associated with coordinate j, we obtain graph $G \cup E(e - e^j)$, which is also chordal. The set of $m^c + 1$ vectors $\{e\} \cup \{e - e^1, e - e^2, \ldots, e - e^{m^c}\}$ is affinely independent and contained in the set $X(G)$, and so it follows that $\text{conv}(X(G))$ is a full-dimensional polytope.

For (b), let $f \in E^c$, and notice that the set of m^c vectors $\{e\} \cup \{e - e^{f'} : f' \in E^c \backslash \{f\}\}$ is affinely independent and satisfies $x_f = 1$. □

6. Cycle Graph Facets

We restrict our attention now to *cycle graphs*—that is, graphs consisting of a single cycle. Cycle graphs are the building blocks of computational methodologies for the MCCP, because finding a chordal completion of a graph naturally concerns identifying chordless cycles and eliminating them by adding chords. We present four classes of facet-defining inequalities for cycle graphs in this section.

Let $G = (V, E)$ be a cycle graph associated with a k-vertex chordless cycle $C = (v_0, \ldots, v_{k-1})$—that is, $V = V(C)$ and $E = \xi(C)$. Assume all additions and subtractions involving indices of vertices are modulo-k. The proofs presented in this section show only the validity of the inequalities; arguments proving that they are facet defining for cycle graphs are presented in Section EC.1 of the online supplement.

Proposition 2. *Let $G = (V, E)$ be a cycle graph associated with cycle $C = (v_0, \ldots, v_{k-1})$, $k \geq 4$. The chordal inequality (I1) associated with C, which in this case simplifies to*

$$\sum_{f \in \iota(C)} x_f \geq |C| - 3,$$

is valid and facet defining for $\text{conv}(X(G))$. □

The proof of the validity of the inequality in Proposition 2 follows directly from Lemma 1.

Proposition 3. *If $k \geq 4$, the inequality*

$$x_{\{v_{i-1}, v_{i+1}\}} + \sum_{f : v_i \in f, \{v_{i-1}, v_{i+1}\} \cap f = \emptyset} x_f \geq 1, \quad \text{for all } i \in \{1, \ldots, k\} \tag{I2}$$

is valid and facet defining for $\text{conv}(X(G))$.

Proof. Suppose that (I2) is violated by some $x \in \text{conv}(X(G))$—that is, that for some

$$\{v_{i-1}, v_i, v_{i+1}\} \subset V,\ x_{\{v_{i-1}, v_{i+1}\}} + \sum_{f : v_i \in f, \{v_{i-1}, v_{i+1}\} \cap f = \emptyset} x_f = 0.$$

As $k \geq 4$, a shortest path P from v_{i-1} to v_{i+1} in $G(x)$ that does not include v_i traverses at least two edges. The sequence defined by the concatenation of P with (v_{i-1}, v_i, v_{i+1}) thus defines a k'-chordless cycle of $G(x)$ for $k' \geq 4$, a contradiction. □

For the next proposition, some additional notation is in order. For any two vertices v_i and v_j with $i < j$, let $d_C(v_i, v_j) := \min\{j - i, k - j + i\}$ be the distance between v_i and v_j in C, and assume $d_C(v_i, v_j) := d_C(v_j, v_i)$ if $i > j$.

Proposition 4. *If $k \geq 5$, the inequality*

$$\sum_{f \in \{\{v_i, v_j\} \in E^c : d_C(v_i, v_j) = 2\}} x_f \geq 2 \tag{I3}$$

is valid and facet defining for $\text{conv}(X(G))$.

Proof. Inequality (I3) states that at least two out of the $|C|$ pairs of vertices of distance 2 must appear in any chordal completion of G. Given completion F of G, without loss of generality, let $f' = \{v_0, v_{j_1}\}$ be the edge of $\iota(C)$ that connects the "closest" vertices with respect to d_C. If $j_1 \geq 3$, then $C' = (v_0, v_1, \ldots, v_{j_1})$ is a chordless cycle in $G \cup F$, a contradiction; therefore, $j_1 = 2$.

As $k \geq 5$, $C' = (v_0, v_2, v_3, \ldots, v_{k-1})$ is a cycle in $G \cup F$ with at least four vertices, so at least one edge of $\iota(C')$ must be present in $G \cup F$. Let $f'' = \{v'_i, v'_{j_2}\}$ be the edge of $\iota(C')$ that connects the "closest" vertices with respect to $d_{C'}$. An argument similar to the one used to show $j_1 = 2$ shows that f'' connects two vertices of distance 2 in C', so we have two cases to analyze. First, if $f'' \neq \{v_0, v_3\}$, the result follows directly. Otherwise, we either have $|C'| = 4$, in which case $d_{C'}(v_0, v_3) = d_C(v_0, v_3) = 2$, as desired, or we have a chordless cycle

$C'' = (v_0, v_3, \ldots, v_{k-1})$ in $G \cup F$ with at least four vertices, on which we can apply the same arguments; as a sequence of cycles emerging from this construction will lead to a cycle of length 4, the result holds. □

Proposition 5. *If $k \geq 5$, the inequality*

$$\sum_{f \in \iota(C) \setminus \{\{v_{j-1}, v_{j+1}\}, \{v_j, v_i\}\}} x_f \geq |C| - 4, \quad \text{for all } i, j \in \{1, \ldots, k\}, \; d_C(v_j, v_i) \geq 2, \tag{I4}$$

is valid and facet defining for conv($X(G)$).

Proof. Given vertices v_i and v_j such that $d_C(v_j, v_i) \geq 2$, Inequality (I4) enforces the inclusion of at least $|C| - 4$ edges of $\iota(C) \setminus \{\{v_{j-1}, v_{j+1}\}, \{v_j, v_i\}\}$ in any chordal completion of G. Without loss of generality, let $j = 0$ and let i be any value in $[2, k-3]$. Suppose by contradiction that there exists $x^0 \in X(G)$ such that

$$\sum_{f \in \iota(C) \setminus \{\{v_{k-1}, v_1\}, \{v_0, v_i\}\}} x^0_f < |C| - 4. \tag{1}$$

By Lemma 1, we have that

$$\sum_{f \in \iota(C)} x^0_f \geq |C| - 3.$$

This implies that $x^0_{v_{k-1}, v_1} = x^0_{v_0, v_i} = 1$, for otherwise Inequality (1) would be violated. Thus, the sequences $C^1 = (v_0, v_1, \ldots, v_i)$ and $C^2 = (v_0, v_i, v_{i+1}, \ldots, v_{k-1})$ are cycles in $G(x^0)$. Again, by Lemma 1, at least $|C^\ell| - 3$ fill edges must be present in $\iota(C^\ell)$, $\ell = 1, 2$. This is only possible if at least $i - 2$ edges of $\iota(C^1)$ and at least $|C| - i - 2$ edges of $\iota(C^2)$ belong to the set of fill edges described by x^0. As $\iota(C^1) \cap \iota(C^2) = \emptyset$ and $\iota(C^1) \cup \iota(C^2) \subseteq \iota(C)$, we have that

$$\sum_{f \in \iota(C) \setminus \{\{v_{i-1}, v_{i+1}\}, \{v_0, v_i\}\}} x^0_f \geq \sum_{f \in \text{int}(C^1)} x^0_f + \sum_{f \in \text{int}(C^2)} x^0_f \geq |C| - 4,$$

thus contradicting Inequality (1).

Example 2. Let G be a cycle graph associated with six-cycle $C = (v_0, v_1, v_2, v_3, v_4, v_5)$. An example of Inequality (I2) with $i = 1$ is

$$x_{\{v_0, v_2\}} + (x_{\{v_1, v_3\}} + x_{\{v_1, v_4\}} + x_{\{v_1, v_5\}}) \geq 1,$$

which enforces the inclusion of at least one edge $f = \{v_1, v_k\}$, $k \in \{3, 4, 5\}$, for every chordal completion without edge (v_0, v_2).

Inequality (I3) translates to

$$x_{\{v_5, v_1\}} + x_{\{v_0, v_2\}} + x_{\{v_1, v_3\}} + x_{\{v_2, v_4\}} + x_{\{v_3, v_5\}} + x_{\{v_4, v_0\}} \geq 2,$$

which enforces that at least two of the six pairs of vertices that are separated by one vertex must appear in any chordal completion of G. Finally, Inequality (I4) for $i = 4$ and $j = 1$ is given by

$$x_{\{v_0, v_3\}} + x_{\{v_0, v_4\}} + x_{\{v_1, v_3\}} + x_{\{v_1, v_5\}} + x_{\{v_2, v_4\}} + x_{\{v_2, v_5\}} + x_{\{v_3, v_5\}} \geq 2,$$

which enforces that at least two of the edges in $\iota(C) \setminus \{\{v_0, v_2\}, \{v_1, v_4\}\}$ must be included in any chordal completion of G.

7. General Polyhedral Properties

This section provides theoretical insights into the polyhedral structure of the MCCP polytope. The first result, provided in Theorem 2, shows that any inequality proven to be valid on an induced subgraph can be extended into a valid inequality for the original graph. This result is relevant because it shows that finding valid inequalities/facets on particular substructures, such as cycles, helps in the generation of valid inequalities for larger graphs containing these substructures. Theorem 3 shows how facets for a cycle graph can be lifted to inequalities that, in turn, are facets for its subgraphs. This leads to the result in Corollary 1 concerning when the Inequalities (I1) in their general form are facet defining. The final result of the section, Theorem 4, proves and describes how facets for small cycles can be lifted to facets of larger cycles.

First, we show a lemma that will be used in the proof of Theorem 2.

Lemma 2. *If $G = (V, E)$ is chordal, then $G[W]$ is chordal for any $W \subseteq V$.*

Proof. It follows since a chordless cycle C in $G[W]$ must be a chordless cycle in G. □

Theorem 2. *Let $G = (V, E)$ be an arbitrary graph and $W \subseteq V$ be any subset of vertices. If $a'x \geq b$ is a valid inequality for $X(G[W])$, then $ax \geq b$ is a valid inequality for $X(G)$, where $a_f = a'_f$ if $f \in E^c(G[W])$ and $a_f = 0$ otherwise.*

Proof. By way of contradiction, suppose that $a'x \geq b$ is valid for $X(G[W])$ and let F be a chordal completion of G such that $ax(F) < b$. From the construction of a, we have $ax(F) = a'x(F \cap E^c(G[W])) < b$. By Lemma 2, $G[W] + F \cap E^c(G[W])$ must be chordal, and, consequently, we must have $a'x(F \cap E^c(G[W])) \geq b$, thus establishing a contradiction. □

We now present a result that goes in the opposite direction of Theorem 2. Namely, it shows how facet-defining inequalities for a cycle graph G can be transformed into facet-defining inequalities for subgraphs of G; note that subgraphs of cycle graphs consist of collections of paths. This result allows us to show that Inequality (I1) is facet defining for subgraphs of cycle graphs.

Theorem 3. *Let $G' = (V, E')$ be a cycle graph associated with the cycle $C = (v_0, v_1, \ldots, v_{k-1}) \in \mathcal{C}$ and $G = (V, E)$ be*

a subgraph of G' such that $G' = G \cup F_G(C)$, $F_G(C) = E' \setminus E$. If $ax \geq b$ is facet defining for $\text{conv}(X(G'))$, *$a \geq \mathbf{0}$, and $a' \in \mathbb{R}^{|E^c|}$, with $a'_f = a_f$ iff $f \in E^c \setminus F_G(C)$ and $a'_f = 0$ otherwise, the inequality*

$$a'x \geq b\left(\sum_{f \in F_G(C)} x_f - |F_G(C)| + 1\right),$$

is facet defining for $\text{conv}(X(G))$. □

Theorem 3 (proved in Section EC.2 of the online supplement) immediately leads to the following result:

Corollary 1. *For any graph $G = (V, E)$, and for any sequence $C \in \mathcal{C}$ such that $\iota(C) \cap E = \emptyset$, the chordal Inequality (I1) is facet defining for the MCCP polytope of $G[V(C)]$.*

Example 3. Consider the graph G in Figure 2(a); solid lines represent graph edges in this example. As in the statement of Theorem 3, we have $C = (v_0, v_1, v_2, v_3, v_4)$, $E(G) = \{\{v_0, v_1\}, \{v_3, v_4\}\}$, and $F_G(C) = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_4, v_0\}\}$. Graph $G \cup F(C)$ is five-chordless cycle. One facet-defining inequality for this cycle, according to Proposition 2, is the simplified version of the chordal inequality, given by

$$x_{\{v_0,v_2\}} + x_{\{v_0,v_3\}} + x_{\{v_1,v_3\}} + x_{\{v_1,v_4\}} + x_{\{v_2,v_4\}} \geq 2.$$

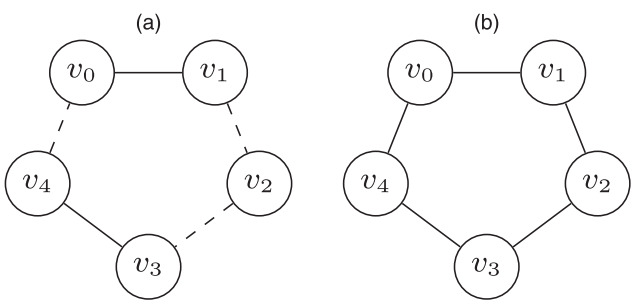
Corollary 1 stipulates that the inequality below is facet defining for G:

$$\begin{aligned} x_{\{v_0,v_2\}} + x_{\{v_0,v_3\}} + x_{\{v_1,v_3\}} + x_{\{v_1,v_4\}} + x_{\{v_2,v_4\}} \\ \geq 2 \cdot (x_{\{v_1,v_2\}} + x_{\{v_2,v_3\}} + x_{\{v_4,v_0\}} - 3 + 1). \end{aligned}$$

By Theorem 2, these inequalities will also be valid even if G is a subgraph of a larger graph.

We now define a method for lifting facet-defining inequalities defined on smaller cycles into facet-defining inequalities for large cycles. This is done by considering the inclusion of chords into the inequalities, which reveals a lifting property of MCCPs that can be used to strengthen known inequalities. We present this result in Theorem 4, which is proved in Section EC.2 of the online supplement.

Figure 2. Subgraph of a Chordless Cycle Graph



Notes. (a) A graph G with $V = \{v_0, v_1, v_2, v_3, v_4\}$ and $E = \{\{v_0, v_1\}, \{v_3, v_4\}\}$. Solid lines represent graph edges and dashed lines are fill edges whose addition to G makes a chordless cycle of length five. (b) Cycle graph with five vertices.

Theorem 4. *Let $G = (V, E)$ be a cycle graph associated with the cycle $C = (v_0, v_1, \ldots, v_{k-1})$ and let $f^* = \{v_s, v_t\} \in E^c$, $0 \leq s < t \leq k-1$, be any chord of C. If the inequality $ax \geq b$, $a \geq \mathbf{0}$, is facet-defining for the MCCP polytope of cycle graph $G' = (V', E')$ associated with the cycle $C' = (v_s, v_{s+1}, \ldots, v_t)$ (i.e., $G' = G[V(C')] + \{f^*\}$), then*

$$a'x \geq b \cdot x_{f^*}$$

is facet defining for $\text{conv}(X(G))$, *where $a'_f = a_f$ if $f \in \iota(C')$ and $a'_f = 0$ otherwise.* □

If $ax \geq b$ is facet defining for any induced subcycle obtained by adding edge f^*, then $ax \geq b \cdot x_{f^*}$ will be facet defining for the original cycle graph.

Example 4. Consider the graph in Figure 2(b), and let $G' = G[\{v_1, v_2, v_3, v_4\}] + \{\{v_1, v_4\}\}$ be a cycle graph associated with cycle $C' = (v_1, v_2, v_3, v_4)$. From Proposition 2, we have that the following chordal inequality is facet defining for $\text{conv}(X(G'))$:

$$\sum_{f \in \iota(C')} x_f = x_{\{v_1,v_3\}} + x_{\{v_2,v_4\}} \geq |C'| - 3 = 1.$$

This inequality can be modified in order to become facet defining for $\text{conv}(X(G))$ through Theorem 4, yielding

$$x_{\{v_1,v_3\}} + x_{\{v_2,v_4\}} \geq x_{\{v_1,v_4\}} \Rightarrow x_{\{v_1,v_3\}} + x_{\{v_2,v_4\}} - x_{\{v_1,v_4\}} \geq 0.$$

8. Solution Method for the MCCP

We now describe a procedure to solve formulation (IPC) for general graphs based on the structural results shown in the previous sections. Because the model has exponentially many constraints, our solution technique is based on a hybrid branch-and-bound procedure that applies separation, lazy-constraint generation and a primal heuristic to the problem.

8.1. Separation Complexity

The problem of separating Inequalities (I1)–(I4) over an *integer* point x^* is equivalent to finding a chordless cycle on the graph completion $G' = (V, E') = G \cup E(x^*)$. In our computational experiments, we used an adapted version of the $O(|V| + |E'|^2)$ procedure proposed by Nikolopoulos and Palios (2007) for this task. Namely, we removed the condition that forces the algorithm to stop as soon as a first cycle is detected, thereby enabling the procedure to potentially identify several cycles every time it is executed. This change does not impact the worst-case complexity of the algorithm. We also note in passing that the recent work by Uno and Satoh (2014) enumerates all chordless cycles of G' in time $O(|E'|)$ per cycle. Nonetheless, the execution of

such procedure at every integer node was far too computationally expensive in our analysis.

The separation problem of (I1)–(I4) over *fractional* points is, however, much more challenging. We state the results below concerning this question.

Theorem 5. *Given a fractional point* $x^* \in [0,1]^{m^c}$:

a. *The separation problem of* (I1) *is NP-complete.*
b. *Inequalities* (I2) *can be separated in* $O(|V|^5)$.
c. *Inequalities* (I3) *can be separated in* $O(|V|^8)$.
d. *The separation problem of* (I4) *is NP-complete.*

Proof. Due to space limitations, we present below only proof sketches for these results. The full version of each proof is presented in Section EC.3 of the online supplement.

a. The proof reduces the *quadratic assignment problem*, a classical and well-studied NP-hard problem, to the *α-quadratic shortest cycle problem* (α-QSCP), introduced in this paper. In the QSCP, we are given a graph $G = (V, E)$ and a quadratic cost function $q : V \times V \to [0,1]$, with $q(u,v) = 0$ if $(u,v) \in E$. A feasible solution of QSCP is a simple chordless cycle $C = (v_1, v_2, \ldots, v_{|C|})$ whose cost is

$$p(C) = \sum_{\{u,v\} \in E(G[C])^C} q(u,v) - |C|.$$

The α-QSCP is the decision version of QSCP in which the goal is to decide whether G has a simple chordless cycle C such that $p(C) < \alpha$. We use a reduction of the quadratic assignment problem to (−3)-QSCP that resembles the ones used by Rostami et al. (2015) for the quadratic shortest path problem. Finally, the −3-QSCP is reduced to the problem of separating Inequality (I1), completing the proof.

b. An auxiliary graph G', a complete digraph on $|V(G)|$ nodes, is constructed for which the separation problem is reduced to finding, for every triple of vertices (v_1, v_2, v_3), the shortest path from v_1 to v_3 that does not include v_2. The number of sequences for which this verification needs to be performed is $O(|V(G)|^3)$, and the identification of such a path can be made in time $O(|V(G)|)^2$.

c. As in b, an auxiliary graph G', specifically a complete digraph on $|V(G)|^2$ nodes, is constructed for which the separation problem is reduced to finding at most $O(|V(G)|^4)$ shortest paths, each of which can be performed in polynomial time.

d. The proof is similar to that of a, except that we use a reduction from −4-QSCP*, a slight variant of −3-QSCP. □

8.2. Heuristic Separation Algorithms

In view of Theorem 5, we tackle model (IPC) by applying a branch-and-bound procedure that alternates between heuristic separation and lazy-constraint generation.

For the lazy constraint generation part, at every integer node of the branching tree, we apply the procedure presented in Section 8.1 to separate at least one violated Inequality (I1)–(I4), similar to a combinatorial Benders methodology (Codato and Fischetti 2006). Proposition 1 ensures that this approach yields an optimal (and feasible) solution to (IPC), because a violated inequality is not found if and only if the resulting graph is chordal.

Nonetheless, adding violated inequalities only at integer points typically yields weak bounds at intermediate nodes of the branching tree. Because a complete separation of fractional points is not viable due to Theorem 5, we consider a heuristic *threshold* procedure. Given a point $x^* \in [0,1]^{m^c}$ and a threshold $\delta \in (0,1)$, let $E^\delta(x) := \{f \in E^c : x_f \geq \delta\}$, we can use the procedure from Section 8.1 to find violating inequalities for the graph $G \cup E^\delta(x^*)$. Such inequalities may not be necessarily violated by x^*, and thus require a (simple) extra verification testing step. The threshold policy does not guarantee that at least one violated inequality is found, but it can be performed efficiently, and, as our numerical experiments indicate, it is a fundamental component for good performance.

8.3. Primal Heuristic

We have also incorporated a primal heuristic to be applied at infeasible integer nodes of the branching tree. The method is based on the state-of-the-art heuristic for the problem, designed by George and Liu (1989). Specifically, the vertices of the graph are sorted in ascending order according to their degree, thereby defining a sequence $S = (v_1, v_2, \ldots, v_{|V|})$. The vertices are then picked one at a time, in the order indicated by S. For each vertex v_i, edges are added to G so that S defines a *perfect elimination ordering*—that is, v_i and its neighbours on set $\{v_{i+1}, v_{i+2}, \ldots, v_{|V|}\}$ induce a clique, which makes G chordal. This procedure has complexity $O(|V|^2 |E|)$. For any integer point $x \in \{0,1\}^{m^c}$ found during the branch-and-bound procedure, if $G' := G \cup E(x)$ is not chordal, we can apply the heuristic of George and Liu (1989) in order to chordalize G' and obtain a feasible solution to the problem. The application of this procedure at the root node ensures that we can identify solutions that are at least as good as those provided by the heuristic.

9. Numerical Experiments

In this section, we present an experimental evaluation of the solution methods introduced in this paper. The experiments ran on an Intel(R) Xeon(R) CPU E5-2640 v2 at 2.80 GHz with 100 GB of RAM. We used the integer programming solver IBM ILOG CPLEX 12.7.1 (IBM ILOG 2017) in all experiments, which were limited to one thread and 3,600 seconds of execution. Experiments that exceeded the memory limit had their time limits reduced by five or 10 minutes in order to enable the extraction of an optimality gap.

9.1. Instances

We considered five families of instances for the experimental evaluation: four structured benchmarks (relaxed caveman graphs, grid graphs, queen graphs, and DIMACS), and one new family derived from an application in knowledge-based systems (RCC-8).

- *Relaxed caveman graphs* (Judd et al. 2011) are used in the analysis of social network models, where small pockets of individuals are tightly connected and have sporadic connections to individuals in other groups. Each instance is randomly generated based on three parameters, $\alpha, \beta \in \mathbb{Z}^+$, and $\gamma \in (0, 1)$. Starting from a set of β disjoint cliques each of size α, each edge is examined, and, with probability γ, one of its endpoints is uniformly at random switched to a vertex belonging to another clique. The structure of relaxed caveman graphs is particularly useful for evaluating algorithms for chordal completions. Namely, the endpoint switches lead to large chordless cycles, thus enforcing the inclusion of several edges in chordal completions. For our experiments, 10 instances of each configuration of $\alpha, \beta \in \{4, 5, 6, 7, 8\}$ and $\gamma = 0.30$ were generated. This set of graphs will be henceforth be referred to as caveman instances.
- *Grid graphs* are graphs whose vertex sets can be partitioned into a set of r *rows* $R_1, \ldots, R_r$ and c *columns* $C_1, \ldots, C_c$. Each pair (r', c') in $[r] \times [c]$ is associated with a single vertex $v_{r',c'} \in R_{r'} \cap C_{c'}$. Vertices $v_{i,j}$ and $v_{k,l}$ are adjacent if and only if either $i = k$ and $|j - l| = 1$, or $j = l$ and $|i - k| = 1$. These graphs have been used in previous computational studies of the MCCP because they contain large chordless cycles (Yüceoğlu 2015).
- *Queen graphs* are extensions of grid graphs with additional edges representing longer hops as well as diagonal movements. More precisely, there exists an edge connecting $v_{i,j}$ to $v_{i',j'}$ if and only if one of the three following conditions is satisfied for some k: (1) $i = i' \pm k$ and $j = j' \pm k$; (2) $i = i'$ and $j = j' \pm k$; or (3) $i = i' \pm k$ and $j = j'$. The configurations of grid graphs and queen graphs used in our experiments are equivalent to those used by Yüceoğlu (2015).
- *DIMACS* is a classical graph coloring benchmark frequently used in computational evaluations of graph algorithms. The data set can be downloaded from http://dimacs.rutgers.edu/challenges/.
- *RCC-8* are graphs derived from real-world region-connection calculus networks (Renz 2002). RCC-8 is the state-of-the-art approach for qualitative topological reasoning with applications, for example, in the Semantic Web (Zhang et al. 2015). In an RCC-8 network, nodes correspond to regions (in some topology) and a connection between nodes represents a relationship the regions share. For instance, regions could be associated with countries and connections with trade agreements.

The main use of RCC-8 networks is to answer qualitative *queries*—for example, what countries may be involved with a particular type of trade. To perform these queries, the networks must be modified by enforcing a notion of consistency (i.e., *path consistency*), which is achieved by chordalizing the graph. The size of the chordal completion plays a critical role on the efficiency of the consistency methods for RCC-8 (Sioutis and Koubarakis 2012, Sioutis 2014).

For our experiments, we extracted subgraphs of an RCC-8 network associated with administrative regions of the United Kingdom. The graph that must be subjected to path consistency was obtained from Nikolaou and Koubarakis (2014) (adm1). For each

Figure 3. (Color online) Cumulative Distribution Plot Evaluating Individual Algorithmic Enhancements

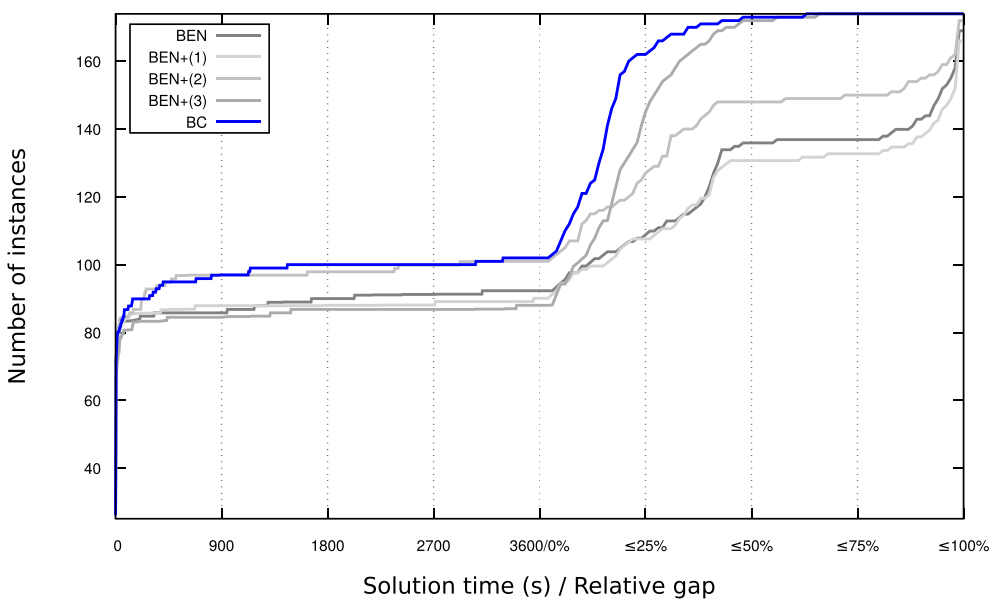


Table 1. Relative Performance of Individual Improvement per Benchmark

		BEN		BEN+(1)		BEN+(2)		BEN+(3)		BC	
Set	Number I	t	% Gap	t	% Gap	t	% Gap	t	% Gap	t	% Gap
Queen	32	$107.2^{(15)}$	32	$120.9^{(15)}$	13	$55.1^{(15)}$	24	$77.5^{(15)}$	31	$109.2^{(15)}$	14
Grid	23	$286.9^{(11)}$	54	$9.4^{(10)}$	25	$68.6^{(13)}$	34	$7.6^{(10)}$	51	$45.1^{(13)}$	13
Caveman	25	$45.0^{(24)}$	22	$47.3^{(24)}$	7	$17.0^{(25)}$	0	$24.8^{(24)}$	38	$5.9^{(25)}$	0
RCC-8	60	$134.6^{(24)}$	59	$175.1^{(22)}$	21	$354.2^{(30)}$	55	$12.2^{(23)}$	61	$346.4^{(31)}$	14
DIMACS	34	$167.0^{(18)}$	83	$91.5^{(17)}$	31	$34.1^{(18)}$	61	$350.9^{(18)}$	83	$64.6^{(18)}$	26
Total	174	$120.4^{(92)}$	56	$84.4^{(88)}$	22	$128.7^{(101)}$	46	$87.6^{(90)}$	57	$138.6^{(102)}$	17

$n \in \{25, 50, 75, 100, 125, 150\}$, we chose n vertices by picking a vertex v uniformly at random, setting $S = \{v\}$, and increasing S by picking vertices that are adjacent to at least one vertex in S uniformly at random until $|S| = n$. Finally, we generated the graph induced by S for our tests. This was repeated 10 times per n.

9.2. Other Approaches

The state-of-the-art techniques to the MCCP reported in the literature are a branch-and-cut approach by Yüceoğlu (2015) and a Benders decomposition approach by Bergman and Raghunathan (2015), henceforth denoted by YUC and BEN, respectively. The complete branch-and-cut algorithm proposed in this paper will be denoted by BC.

YUC employs a branch-and-cut approach based on a polyhedral analysis of a *perfect elimination ordering* (PEO) model for the MCCP. The PEO model is presented in Section EC.4 of the online supplement. YUC incorporates valid inequalities for grid and queen graphs. BEN is the precursor of the approach described in the present work, in that it solves a formulation consisting only of Inequalities (I1) with a pure Benders decomposition approach; that is, BEN solves to optimality an MILP using the current set of Inequalities (I1) found up to the current iteration. If the solution contains no chordless cycles, then it is optimal. Otherwise, a collection of chordless cycles is found, new Inequalities (I1) are added to the model, and the procedure repeats.

Also of interest is to compare our approach with state-of-the-art heuristics in terms of solution quality, thus assessing how significant the differences between exact and heuristic solutions are. We consider

Table 2. Results for Grid Graphs

	Instance		YUC			BC			MDO
Name	$\|V\|$	$\|E\|$	*LB*	*UB*	t	*LB*	*UB*	t	*UB*
grid3_3	9	12	5	**5**	0.01	5	**5**	0.01	**5**
grid3_4	12	17	9	**9**	0	9	**9**	0.01	**9**
grid3_5	15	22	13	**13**	0.02	13	**13**	0.04	**13**
grid3_6	18	27	17	**17**	0.02	17	**17**	0.09	**17**
grid3_7	21	32	21	**21**	0.01	21	**21**	0.12	**21**
grid3_8	24	37	25	**25**	0.02	25	**25**	0.69	**25**
grid3_9	27	42	29	**29**	0.02	29	**29**	0.87	33
grid3_10	30	47	33	**33**	0.03	33	**33**	2.6	37
grid4_4	16	24	18	**18**	1.23	18	**18**	0.58	**18**
grid4_5	20	31	25	**25**	18.11	25	**25**	3.95	**25**
grid4_6	24	38	32.2	**34**	—	34	**34**	71.26	**34**
grid4_7	28	45	39	**41**	—	41	**41**	370.9	**41**
grid4_8	32	52	45.5	52	—	47.97	**50**	—	**50**
grid4_9	36	59	52.5	58	—	52.89	**57**	—	**57**
grid4_10	40	66	59.3	**66**	—	58.58	**66**	—	**66**
grid5_5	25	40	[a]	[a]	[a]	37	**37**	135.32	**37**
grid5_6	30	49	46.2	53	—	47.31	**50**	—	52
grid5_7	35	58	56.9	65	—	56.03	**62**	—	68
grid5_8	40	67	67.5	77	—	63.30	**75**	—	80
grid5_9	45	76	33.3	90	—	70.33	**87**	—	93
grid6_6	36	60	60.9	77	—	57.03	**69**	—	71
grid6_7	42	71	31	94	—	68.22	**86**	—	92
grid7_7	49	84	37	125	—	80.82	**111**	—	119

Notes. Numerical results for the YUC columns were taken directly from Yüceoğlu's thesis (Yüceoğlu 2015). An upper bound presented in boldface indicates that the algorithm found the best-known solution for the instance.
[a]grid5_5 was not included in the results by Yüceoğlu (2015).

Table 3. Results for Queen Graphs

Instance			YUC			BC			MDO
name	$\|V\|$	$\|E\|$	*LB*	*UB*	*t*	*LB*	*UB*	*t*	*UB*
queen3_3	9	28	5	**5**	0	5	**5**	0.01	**5**
queen3_4	12	46	12	**12**	0.01	12	**12**	0.01	**12**
queen3_5	15	67	22	**22**	0.31	22	**22**	0.01	**22**
queen3_6	18	91	36	**36**	1.03	36	**36**	0.07	**36**
queen3_7	21	118	53	**53**	2.17	53	**53**	0.06	**53**
queen3_8	24	148	74	**74**	8.49	74	**74**	2.58	**74**
queen3_9	27	181	98	**98**	15.77	98	**98**	9.37	**98**
queen3_10	30	217	126	**126**	65.91	126	**126**	43.08	**126**
queen4_4	16	76	26	**26**	0.19	26	**26**	0.02	28
queen4_5	20	110	51	**51**	4.54	51	**51**	0.54	53
queen4_6	24	148	83	**83**	16.54	83	**83**	13.25	**83**
queen4_7	28	190	119	**119**	68.22	119	**119**	73.56	121
queen4_8	32	236	164	**164**	636.28	164	**164**	1,129.7	167
queen4_9	36	286	209.8	**217**	—	208.5	218	—	222
queen4_10	40	340	255.5	**278**	—	254.72	279	—	286
queen5_5	25	160	93	**93**	41.03	93	**93**	50.06	94
queen5_6	30	215	144	**144**	185.81	144	**144**	315.97	154
queen5_7	35	275	203.1	**214**	—	200.25	215	—	223
queen5_8	40	340	265.8	**293**	—	261.49	294	—	306
queen5_9	45	410	339.8	393	—	335.27	**392**	—	398
queen5_10	50	485	424.9	501	—	422.31	**497**	—	503
queen6_6	36	290	214.9	232	—	210.13	**231**	—	244
queen6_7	42	371	299.2	351	—	292.59	**340**	—	352
queen6_8	48	458	400.7	481	—	394.47	**461**	—	482
queen6_9	54	551	521.4	622	—	512.35	**615**	—	633
queen6_10	60	650	656.7	**786**	—	642.16	792	—	826
queen7_7	49	476	423.7	520	—	418.85	**502**	—	515
queen7_8	56	588	577.6	710	—	566.06	**684**	—	687
queen7_9	63	707	751.8	935	—	733.14	**905**	—	919
queen7_10	70	833	948.5	1,177	—	924.79	**1,141**	—	1149
queen8_8	64	728	782.1	965	—	767.02	**938**	—	970
queen9_9	81	1,056	[a]	[a]	[a]	1,289.17	**1,641**	—	1664

Notes. Numerical results for the YUC columns were taken directly from Yüceoğlu's thesis (Yüceoğlu 2015). An upper bound presented in boldface indicates that the algorithm found the best-known solution for the instance.

[a]queen9_9 was not included in the results by Yüceoğlu (2015).

the state-of-the-art heuristic developed by George and Liu (1989) (described in Section 8.3), which will henceforth be denoted by MDO.

9.3. Algorithmic Enhancements

We first evaluate the impact of individual algorithmic enhancements on BEN. The following enhancements are considered: (1) using all inequalities (I1)–(I4); (2) separating the inequalities at fractional nodes based on our polyhedral results; and (3) invoking MDO as a primal heuristic. BC is an implementation where all enhancements are applied—that is, BEN + (1) + (2) + (3).

Figure 3 presents a cumulative distribution plot of performance considering all instances classes. The horizontal axis is divided into two areas: The left portion denotes solution time (in seconds), and the right portion denotes relative optimality gap (in this case, computed as $(ub - lb)/(ub + 10^{-8})$, where *ub* and *lb* represent the resulting upper and lower bounds, respectively). The vertical axis denotes the number of solutions with that solution time or relative gap. Note that instances solved in up to 3,600 s have a final gap of 0%, and hence the curves are nondecreasing. Furthermore, each instance of *caveman* was counted as 0.1 because of its randomized process.

The plot indicates that separating at fractional nodes [BEN + (2)] significantly improves the optimality proof time for instances that were solved within the time limit. Adding a primal heuristic [BEN + (3)] was crucial in obtaining better relative gaps, especially because MDO typically provides good solutions quickly, as exhibited later in this section. In contrast, additional cuts without fractional separation [BEN + (1)] did not provide significant enhancements. The complete method, BC, outperformed all other approaches both in terms of solution times and relative gaps, thus suggesting a positive effect of combining the individual enhancements. The same behavior was observed when plotting each benchmark separately. Other possible combinations were dominated by BC as well.

Table 4. Results for DIMACS Graphs

Instance			YUC			BC			MDO
Name	$\|V\|$	$\|E\|$	*LB*	*UB*	*t*	*LB*	*UB*	*t*	*UB*
anna	138	493	47	**47**	1,386.04	47	**47**	0.64	**47**
david	87	406	59.5	65	—	64	**64**	1.84	66
games120	120	638	496.4	1,626	—	902.48	**1,452**	—	1,513
huck	74	301	5	**5**	2.92	5	**5**	0.03	9
jean	80	254	16	**16**	6.13	16	**16**	0.06	19
miles250	128	387	45.7	61	—	53	**53**	4.95	61
miles500	128	1,170	196.4	447	—	337.68	**404**	—	446
miles750	128	2,113	352.1	954	—	471	**471**	812.59	723
myciel3	11	20	10	**10**	0	10	**10**	0	**10**
myciel4	23	71	46	**46**	0.06	46	**46**	0.01	**46**
myciel5	47	236	189.7	197	—	196	**196**	17.04	197
1-FullIns_3	30	100				80	**80**	1.22	**80**
1-FullIns_4	93	593				652.75	**776**	—	839
1-Insertions_4	67	232				294.99	**364**	—	394
2-FullIns_3	52	201				238.65	**248**	—	273
2-Insertions_3	37	72				87.46	**99**	—	103
2-Insertions_4	149	541				599.21	**1,585**	—	1,588
3-FullIns_3	80	346				427.18	**575**	—	661
3-Insertions_3	56	110				124.96	**191**	—	198
4-FullIns_3	114	541				639.86	**1,110**	—	1,274
4-Insertions_3	79	156				170.54	**322**	—	331
DSJC125.1	125	736				1,714.13	**2,599**	—	2,618
DSJC125.5	125	3,891				2,367.78	**3,240**	—	**3,240**
DSJC125.9	125	6,961				583.08	**734**	—	**734**
miles1000	128	3,216				535	**535**	286.98	700
miles1500	128	5,198				218	**218**	1.13	308
mug100_1	100	166				64	**64**	0.48	91
mug100_25	100	166				64	**64**	0.90	93
mug88_1	88	146				56	**56**	0.33	82
mug88_25	88	146				56	**56**	0.68	84
myciel6	95	755				730.97	**753**	—	**753**
r125.1	125	209				11	**11**	1.53	15
r125.1c	125	7,501				207	**207**	33.83	**207**
r125.5	125	3,838				1,006.78	**1,212**	—	1,231

Notes. Numerical results for the YUC columns were taken directly from Yüceoğlu's thesis (Yüceoğlu 2015). An upper bound presented in boldface indicates that the algorithm found the best-known solution for the instance.

In order to verify whether an individual enhancement leads to a statistically significant improvement in performance, a two-tailed paired *t*-test was used. Specifically, the null hypothesis indicates whether the solution times or gaps are equivalent with or without an enhancement. We first considered the solution times, restricting to instances that were solved within 3,600 s for all methodologies. For algorithms BEN + (1), BEN + (2), BEN + (3), and BC, the tests resulted in a *p*-value of 0.57, 0.00024, 0.088, and 7.6×10^{-5}, respectively, in comparison with BEN. Notice that this is consistent with Figure 3, because only BEN + (2) and BC provided significant and consistent improvements to BEN. For the relative optimality gap, we consider all tested instances. The *p*-values for BEN + (1), BEN + (2), BEN + (3), and BC were then 0.016, 5.7×10^{-7}, 2.0×10^{-12}, and 2.5×10^{-14}, respectively. This is again consistent with Figure 3 and shows that the relative gaps for BEN + (3) and BC are significantly superior to BEN.

Table 1 provides further details on the impact of the enhancements on each data set. Column #*I* reports the total number of instances in the data set; columns *t* provides the average solution time for instances that were solved within 3,600 seconds, with the number in parentheses representing the number of such instances; and columns %*Gap* provides the average gap for instances that were not solved within the time limit. BC outperforms the other configurations, with respect to both the relative gap and the number of instances solved within the time limit. There were no instances solved by other configurations that BC was unable to solve.

Cut Analysis. We also experiment with enabling and disabling distinct combinations of valid Inequalities (I2)–(I4) in the presence of the primal heuristic and separation at fractional nodes. BC had a higher average running time than the algorithms with no extra

Table 5. Comparison Between MDO and BC on Caveman Graphs

α	β	BC % Gap	BC t	BC UB	MDO UB	% Decrease	Minimum % decrease	Maximum % decrease
4	4	0.0	0.0	0.9	1.4	15.0	0.0	50.0
4	5	0.0	0.0	1.7	2.8	35.11	0.0	75.0
4	6	0.0	0.0	5.3	7.4	42.75	0.0	66.66
4	7	0.0	0.03	13.2	17.3	25.1	8.33	56.25
4	8	0.0	0.07	19.0	26.3	29.27	6.66	64.28
5	4	0.0	0.0	1.6	1.8	6.25	0.0	50.0
5	5	0.0	0.0	1.7	4.4	51.77	12.5	80.0
5	6	0.0	0.01	7.7	10.3	40.3	0.0	66.66
5	7	0.0	0.17	15.5	21.7	35.67	0.0	54.54
5	8	0.0	0.27	25.1	37.1	34.76	13.79	61.9
6	4	0.0	0.0	0.6	1.7	32.14	0.0	71.42
6	5	0.0	0.0	4.1	8.4	54.92	16.66	80.0
6	6	0.0	0.15	12.2	16.9	35.41	0.0	61.53
6	7	0.0	0.35	18.1	24.2	25.28	6.12	50.0
6	8	0.0	1.47	28.9	43.0	35.76	8.82	59.25
7	4	0.0	0.0	1.1	2.5	50.83	0.0	75.0
7	5	0.0	0.0	3.6	6.5	42.85	0.0	83.33
7	6	0.0	0.32	17.1	23.6	35.79	7.31	38.46
7	7	0.0	2.05	24.8	35.4	37.63	9.37	55.81
7	8	0.0	35.53	55.9	74.1	27.61	11.39	70.96
8	4	0.0	0.0	0.3	3.3	70.83	33.33	75.0
8	5	0.0	0.01	5.5	9.9	58.66	0.0	80.0
8	6	0.0	3.96	14.6	23.3	39.92	0.0	92.85
8	7	0.0	3.76	29.7	45.0	37.64	8.0	81.25
8	8	0.0	101.44	52.6	69.8	26.67	13.88	56.81

inequalities or with a single family of cuts (I2)–(I4). Nevertheless, BC solved to optimality either one or two instances more than each of the other algorithms; in particular, BC solved all instances that were solved by the other combinations. Average relative gaps varied by at most 1% between combinations. Finally, two-tailed paired t-tests indicated that there was no significant difference in solution times or relative gaps between combinations. We therefore use all Inequalities (I2)–(I4) in the subsequent experiments.

9.4. Comparison with Other Approaches

This section provides a comparison of BC with YUC, as well as with the MDO heuristic. We first focus on the instances reported upon in Yüceoğlu (2015) and Bergman and Raghunathan (2015)—grid, queen, and DIMACS graphs—and compare solution times and objective function bounds. The reported numbers for YUC were obtained directly from Yüceoğlu (2015), which were run with IBM ILOG CPLEX 12.2 and a processor with similar clock (2.53 GHz), but using the parallel version of the solver, with four cores, not with one core, as was set in our experiments.

The data for grid graphs, queen graphs, and DIMACS graphs are presented in Tables 2–4, respectively. These tables report, for each instance, the number of vertices, the number of edges, and, for all algorithms, the resulting lower bounds, upper bounds, and solution times (in seconds if solved to optimality in 3,600 s, or a "—" mark otherwise). An upper bound presented in boldface indicates that the algorithm found the best-known solution for the instance.

As a summary of our results, BC found the best known solutions in 85 out of all 89 of these instances and improved upon the best lower bound in 14 of the 64 instances (13 by more than the least integer greater than the bound) that have previously been reported in the literature. In particular, the optimality gap in six of the instances was closed for the first time. We now discuss the results for each class of graph in detail.

For grid graphs, Yüceoğlu (2015) enhances PEO with cuts tailored for graphs containing grid structures, so these instances are particularly well suited for YUC. The results are presented in Table 2. BC always found the best-known solutions, and only in four cases out of 23 the relaxation bound for YUC outperformed that of BC (three by more than the last integer greater than the bound). Also, BC improved upon the solutions by MDO in nine instances, proving optimality for three new cases.

Next, Table 3 reports on queen graphs. As previously mentioned, these instances are also well-suited to YUC because of their grid-like structures. The results show that BC typically delivers better upper bounds, whereas YUC frequently returns smaller optimality gaps and solution times for harder instances. We remark that different configurations of BC [e.g., the one using only family of cuts (I4)] delivered the best-known upper bounds for all instances of this class of graphs.

Table 6. Comparison Between MDO and BC on RCC-8 Graphs

n	BC % Gap	BC t	BC UB	MDO UB	% decrease	Minimum % decrease	Maximum % decrease
25	0.0	0.01	10.6	10.7	1.0	0.0	10.0
50	0.0	16.56	30.4	31.4	3.36	0.0	8.1
75	3.27	1,362.64	72.4	77.6	6.97	0.0	30.3
100	7.77	2,934.65	101.5	109.1	7.28	0.0	27.84
125	14.08	3,600.0	148.6	157.8	5.98	0.79	17.39
150	17.23	3,600.0	190.8	202.1	5.59	2.96	9.5

Table 4 reports on DIMACS graphs. The 11 instances above the double horizontal line are those reported on in Yüceoğlu (2015), whereas the others are the remaining graphs in the benchmark set consisting of fewer than 150 vertices. Our results show that instances of the first group were solved orders of magnitude faster by BC, and, for those in which YUC was not able to prove optimality, substantially better objective function bounds were obtained. In particular, BC was able to close the optimality gap of four instances of this data set that were still open: david, miles250, miles750, and myciel5. These results can be explained by the fact that DIMACS graphs do not necessarily have grid-like structures, which makes them more challenging for YUC. For the remaining instances, nine were solved to optimality, and for many of the other instances, the best solutions obtained by BC used substantially fewer fill edges than those obtained by the traditional heuristic MDO. Namely, BC improved upon the solution obtained by MOD in 26 of the 34 DIMACS graphs.

We now evaluate the solution quality provided by BC on the remaining data sets. Table 5 presents the results for relaxed caveman graphs. The columns indicate the parameters α and β, and, for each configuration, the average optimality gap, solution time, upper and lower bounds provides by BC, as well as the average upper bound provided by MDO and the average, minimum, and maximum percentage decrease in the upper bound from MDO to BC over each individual instance in that configuration. All instances have been solved to optimality by BC, so all gaps are equal to zero. As MDO runs in under a hundredth of a second in all cases, running times are not reported. The results show the advantage of seeking optimal solutions. The heuristic can be far from the optimum (up to 70% on average for some configurations), showing a trade-off between computational effort and solution quality.

Finally, Table 6 shows the detailed results for RCC-8 graphs. The results are aggregated by the number of vertices of the instances (n), where the first column indicates n and the remaining columns are presented as they are in Table 5. We observed that MDO, which is typically used for RCC-8, was highly effective for $n \in \{25, 50\}$. For larger instances with $n \geq 75$, we saw that BC can identify substantially better solutions. The reduction in the number of edges in the obtained solution can be as high as 30% and is on average approximately 5%. Finding the solutions requires substantial computational time, but because this application does not require expedience in identifying solutions, using BC is particularly attractive.

10. Conclusion

In this paper, we described a new mathematical programming formulation for the MCCP and investigated some key properties of its polytope. The constraints used in our model correspond to lifted inequalities of induced cycle graphs. Our theoretical results show that this lifting procedure can be generalized to derive other facets of the MCCP polytope of cycle graphs. Finally, we proposed a hybrid solution technique that considers both a lazy-constraint generation and a heuristic separation method based on a threshold rounding and also presented a simple primal heuristic for the problem. A numerical study indicates that our approach substantially outperforms existing methods, often by orders of magnitude.

Acknowledgments

The authors thank department editor Yinyu Ye, the associate editor, and two referees for helpful comments during the review process. The authors also acknowledge Renato L. de F. Cunha for providing support for the numerical experiments.

References

Agrawal A, Klein P, Ravi R (1993) Cutting down on fill using nested dissection: Provably good elimination orderings. George A, Gilber JR, Liu JWH, eds. *Graph Theory and Sparse Matrix Computation* (Springer, New York), 31–55.

Beeri C, Fagin R, Maier D, Yannakakis M (1983) On the desirability of acyclic database schemes. *J. ACM*. 30(3):479–513.

Bergman D, Raghunathan AU (2015) A Benders approach to the minimum chordal completion problem. Pesant G, ed. *Principles and Practice of Constraint Programming—CP 2015*, Lecture Notes in Computer Science, vol. 6308 (Springer, Berlin), 47–64.

Berry A, Bordat JP, Heggernes P, Simonet G, Villanger Y (2006) A wide-range algorithm for minimal triangulation from an arbitrary ordering. *J. Algorithms* 58(1):33–66.

Berry A, Heggernes P, Simonet G (2003) The minimum degree heuristic and the minimal triangulation process. Bodlaender H, ed. *Graph-Theoretic Concepts in Computer Science*, Lecture Notes in Computer Science, vol. 2880 (Springer, Berlin), 58–70.

Bliznets I, Cygan M, Komosa P, Mach L, Pilipczuk M (2016) Lower bounds for the parameterized complexity of minimum fill-in and

other completion problems. *Proc. 27th Annual ACM-SIAM Sympos. Discrete Algorithms* (Society for Industrial and Applied Mathematics, Philadelphia), 1132–1151.

Bouchitté V, Todinca I (2001) Treewidth and minimum fill-in: Grouping the minimal separators. *SIAM J. Comput.* 31(1):212–232.

Cao Y, Marx D (2016) Chordal editing is fixed-parameter tractable. *Algorithmica* 75(1):118–137.

Cao Y, Sandeep RB (2016) Minimum fill-in: Inapproximability and almost tight lower bounds. *Proc. 28th Annual ACM-SIAM Sympos. Discrete Algorithms* (Society for Industrial and Applied Mathematics, Philadelphia), 875–880.

Chang MS (1996) Algorithms for maximum matching and minimum fill-in on chordal bipartite graphs. Asano T, Igarashi Y, Nagamochi H, Miyano S, Suri S, eds. *Algorithms and Computation*, Lecture Notes in Computer Science, vol. 1178 (Springer, Berlin), 146–155.

Chung F, Mumford D (1994) Chordal completions of planar graphs. *J. Combin. Theory Ser. B*. 62(1):96–106.

Codato G, Fischetti M (2006) Combinatorial Benders' cuts for mixed-integer linear programming. *Oper. Res.* 54(4):756–766.

Feremans C, Oswald M, Reinelt G (2002) A y-formulation for the treewidth. Technical report, Heidelberg University, Heidelberg, Germany.

Fomin FV, Philip G, Villanger Y (2013) Minimum fill-in of sparse graphs: Kernelization and approximation. *Algorithmica* 71(1):1–20.

Fomin FV, Villanger Y (2012) Subexponential parameterized algorithm for minimum fill-in. *Proc. 23rd Annual ACM-SIAM Sympos. Discrete Algorithms* (Society for Industrial and Applied Mathematics, Philadelphia), 1737–1746.

Fulkerson DR, Gross OA (1965) Incidence matrices and interval graphs. *Pacific J. Math.* 15(3):835–855.

Garey MR, Johnson DS (1979) *Computers and Intractability: A Guide to the Theory of NP-Completeness* (W. H. Freeman & Co., New York).

George A, Liu WH (1989) The evolution of the minimum degree ordering algorithm. *SIAM Rev.* 31(1):1–19.

Grone R, Johnson CR, Sá EM, Wolkowicz H (1984) Positive definite completions of partial hermitian matrices. *Linear Algebra Appl.* 58(0):109–124.

Heggernes P (2006) Minimal triangulations of graphs: A survey. *Discrete Math.* 306(3):297–317.

IBM ILOG (2017) *CPLEX Optimization Studio 12.7.1 User Manual* (IBM ILOG, New York).

Judd S, Kearns M, Vorobeychik Y (2011) Behavioral conflict and fairness in social networks. Chen N, Elkind E, Koutsoupias E, eds. *Internet and Network Economics*, Lecture Notes in Computer Science, vol. 7090 (Springer, New York), 242–253.

Kaplan H, Shamir R, Tarjan RE (1999) Tractability of parameterized completion problems on chordal, strongly chordal, and proper interval graphs. *SIAM J. Comput.* 28(5):1906–1922.

Kim S, Kojima M, Mevissen M, Yamashita M (2011) Exploiting sparsity in linear and nonlinear matrix inequalities via positive semidefinite matrix completion. *Math. Programming* 129(1): 33–68.

Kloks T, Bodlaender H, Müller H, Kratsch D (1993) *Computing Treewidth and Minimum Fill-In: All You Need Are the Minimal Separators* (Springer, Berlin), 260–271.

Kloks T, Kratsch D, Müller H (1999) Approximating the bandwidth for asteroidal triple-free graphs. *J. Algorithms* 32(1):41–57.

Kloks T, Kratsch D, Wong C (1998) Minimum fill-in on circle and circular-arc graphs. *J. Algorithms* 28(2):272–289.

Kratsch D, Müller H (2009) On a property of minimal triangulations. *Discrete Math* 309(6):1724–1729.

Lauritzen SL, Spiegelhalter DJ (1990) Local computations with probabilities on graphical structures and their application to expert systems. Shafer G, Pearl J, eds. *Readings in Uncertain Reasoning* (Morgan Kaufmann Publishers Inc., San Francisco), 415–448.

Mezzini M, Moscarini M (2010) Simple algorithms for minimal triangulation of a graph and backward selection of a decomposable Markov network. *Theoret. Comput. Sci.* 411(7–9):958–966.

Nakata K, Fujisawa K, Fukuda M, Kojima M, Murota K (2003) Exploiting sparsity in semidefinite programming via matrix completion II: Implementation and numerical results. *Math. Programming* 95(2):303–327.

Natanzon A, Shamir R, Sharan R (2000) A polynomial approximation algorithm for the minimum fill-in problem. *SIAM J. Comput.* 30(4):1067–1079.

Nikolaou C, Koubarakis M (2014) Fast consistency checking of very large real-world RCC-8 constraint networks using graph partitioning. *Proc. 28th AAAI Conf. A.I.* (Association for the Advancement of Artificial Intelligence, Menlo Park, CA), 2724–2730.

Nikolopoulos SD, Palios L (2007) Detecting holes and antiholes in graphs. *Algorithmica* 47(2):119–138.

Ohtsuki T (1976) A fast algorithm for finding an optimal ordering for vertex elimination on a graph. *SIAM J. Comput.* 5(1):133–145.

Ohtsuki T, Cheung LK, Fujisawa T (1976) Minimal triangulation of a graph and optimal pivoting order in a sparse matrix. *J. Math. Anal. Appl.* 54(3):622–633.

Parra A, Scheffler P (1995) How to use the minimal separators of a graph for its chordal triangulation. Fülöp Z, Gécseg F, eds. *Automata, Languages and Programming—ICALP 1995*, Lecture Notes in Computer Science, vol. 944 (Springer, Berlin), 123–134.

Parra A, Scheffler P (1997) Characterizations and algorithmic applications of chordal graph embeddings. *Discrete Appl. Math.* 79(1): 171–188.

Parter S (1961) The use of linear graphs in gauss elimination. *SIAM Rev.* 3(2):119–130.

Renz J (2002) The region connection calculus. *Qualitative Spatial Reasoning with Topological Information* (Springer, Berlin), 41–50.

Rollon E, Larrosa J (2011) On mini-buckets and the min-fill elimination ordering. Lee J, ed. *Principles and Practice of Constraint Programming (CP)* (Springer, Berlin), 759–773.

Rose DJ (1972) A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations. Read RC, ed. *Graph Theory and Computing* (Academic Press, Cambridge, MA), 183–217.

Rose DJ, Tarjan RE (1978) Algorithmic aspects of vertex elimination on directed graphs. *SIAM J. Appl. Math.* 34(1):176–197.

Rose DJ, Tarjan RE, Lueker GS (1976) Algorithmic aspects of vertex elimination on graphs. *SIAM J. Comput.* 5(2):266–283.

Rostami B, Malucelli F, Frey D, Buchheim C (2015) On the quadratic shortest path problem. *Internat. Sympos. Experiment. Algorithms* (Springer, New York), 379–390.

Sioutis M (2014) Triangulation versus graph partitioning for tackling large real world qualitative spatial networks. *2014 IEEE 26th Internat. Conf. Tools Artificial Intelligence* (IEEE, Piscataway, NJ), 194–201.

Sioutis M, Koubarakis M (2012) Consistency of chordal rcc-8 networks. *2012 IEEE 24th Internat. Conf. Tools Artificial Intelligence*, vol. 1 (IEEE, Piscataway, NJ), 436–443.

Tarjan RE, Yannakakis M (1984) Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs. *SIAM J. Comput.* 13(3): 566–579.

Uno T, Satoh H (2014) An efficient algorithm for enumerating chordless cycles and chordless paths. *Internat. Conf. Discovery Sci.* (Springer, New York), 313–324.

Vandenberghe L, Andersen MS (2015) Chordal graphs and semidefinite optimization. *Foundations Trends Optim.* 1(4):241–433.

Yannakakis M (1981) Computing the minimum fill-in is NP-Complete. *SIAM J. Algebraic Discrete Methods* 2(1):77–79.

Yüceoğlu B (2015) Branch-and-cut algorithms for graph problems. Unpublished doctoral thesis, Maastricht University, Mastricht, Netherlands.

Zhang C, Zhao T, Li W (2015) *Ontology Data Query in Geospatial Semantic Web* (Springer International Publishing, Cham, Switzerland), 57–87.

David Bergman is an assistant professor of operations and information management in the School of Business at the University of Connecticut. His research interests revolve around the development of novel methodologies for discrete optimization.

Carlos H. Cardonha is a research staff member in the Optimization Under Uncertainty Group at IBM Research—Brazil, Sao Paulo. His primary research interests are in discrete optimization and theoretical computer science, focusing on the development of algorithms based on mixed-integer linear programming, combinatorial optimization, and approximation schemes for operations research problems.

Andre A. Cire is an assistant professor at the Department of Management at University of Toronto, Scarborough, cross-appointed with the Operations Management area at Rotman School of Management. His main research interests include discrete optimization, mathematical programming, dynamic programming, and practical applications of scheduling, routing, and healthcare.

Arvind U. Raghunathan is a senior principal research scientist in the Data Analytics Group at Mitsubishi Electric Research Labs. His research interests are in the development of algorithms for continuous and discrete optimization problems with applications in electric power systems, model-based control, and transportation.